

Finally, it's time to show another important feature of dgoss. Create a file *goss_wait.yaml*, as shown below, in your working directory. Wait for the tests in the *goss_wait.yaml* to be passed before executing the Docker command shown in the last section. This is useful when there is a set of precondition tests to move further. In our case, we ensured that no *ssh* functionality is part of the Spark image, as shown in Figure 6.

```
--- goss_wait.yaml ---
```

```
port:
  tcp:22:
    listening: false
  ip:
    - 0.0.0.0
user:
  sshd:
    exists: false
process:
  sshd:
    running: false
```

Try to change a test in the *goss_wait.yaml* to true and you should see dgoss aborting the test run due to the failure of the *goss_wait.yaml* testing.

Validation of stack runtime of containers through dcgoss

Docker Compose is a popular tool to maintain the lifecycle of a multi-container stack. There is another wrapper created over goss to accept test containers constituting the entire compose stack. Let's jump into it and create *Dockerfile_DCGossDriver* as per the diff shown below, in your current working directory. Now run the command *docker build -f Dockerfile_DCGossDriver . -t dcgoss:0.4.9* in your terminal to build the driver image.

```
8a9
> ARG DCPS_VERSION=2.31.0
16c17
< && curl -sSL "https://github.com/goss-org/goss/releases/download/v${GOSS_VERSION}/dgoss" -o /usr/local/bin/dgoss \
---
> && curl -sSL "https://github.com/goss-org/goss/releases/download/v${GOSS_VERSION}/dcgoss" -o /usr/local/bin/dcgoss \
20a22,23
> && curl -sSL "https://github.com/docker/compose/releases/download/v${DCPS_VERSION}/docker-compose-${uname|tr A-Z a-z}-${DGSPLT}" -o /usr/local/bin/docker-compose \
> && chmod +x /usr/local/bin/docker-compose \
28c31
< CMD ["dgoss", "-h"]
---
> CMD ["dcgoss", "-h"]
```

Next, create the *docker-compose.yaml*, shown below, and *kafka.yaml* to bring up the ZooKeeper+Kafka containers stack and run goss tests over these, respectively.

```
--- docker-compose.yaml ---
```

```
services:
  zookeeper:
    image: 'bitnami/zookeeper:latest'
    ports:
      - '2181:2181'
    environment:
      - ALLOW_ANONYMOUS_LOGIN=yes
  kafka:
    image: 'bitnami/kafka:latest'
    ports:
      - '9092:9092'
    environment:
      - KAFKA_BROKER_ID=1
```

```
INFO: Creating docker container
INFO: Copy goss files into container
Successfully copied 13.7MB to c769e4dc80b033cc2073e3baa21fbf6158d2aac6ec2499b8c256e6cf725009e7:/goss
INFO: Starting docker container
INFO: Container ID: c769e4dc
INFO: Found goss_wait.yaml, waiting for it to pass before running tests
INFO: Sleeping for 10
INFO: Container health
INFO: Running Tests
1..5
ok 1 - Process: java: running: matches expectation: true
ok 2 - Port: tcp6:7077: listening: matches expectation: true
not ok 3 - Port: tcp:8080: listening: Expected false to equal true
ok 4 - Port: tcp6:8080: listening: matches expectation: true
not ok 5 - Port: tcp:7077: listening: Expected false to equal true
INFO: Deleting container
```

Figure 6: *goss_wait* tests that passed for Spark